Práctica 15 — Disparar hacia el ratón

Programación de Videojuegos · FP Lumara · Prácticas previas: 12 y 14

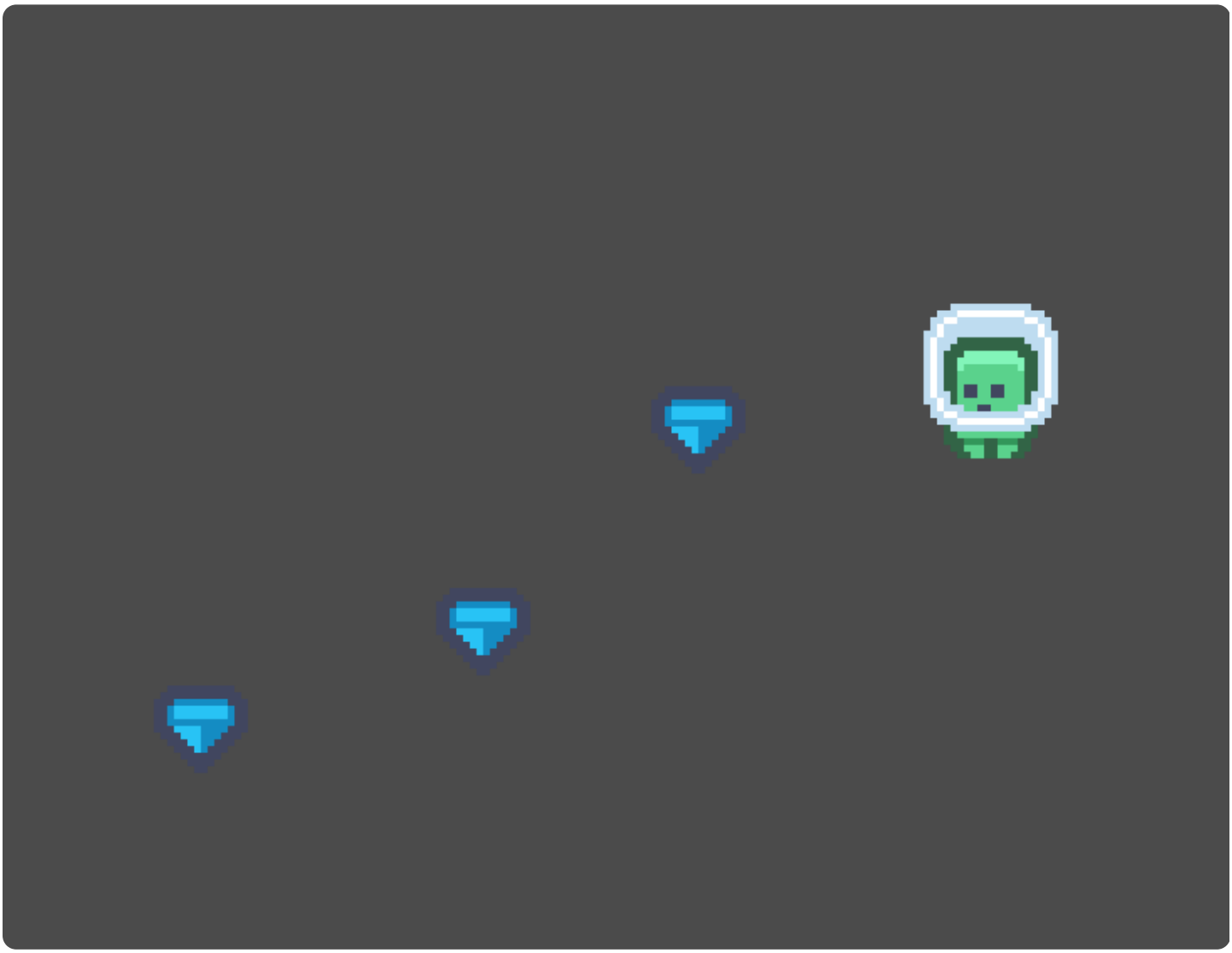
> ***Lo que aprendes hoy: a crear algo con una dirección de partida, y a que se destruya solo cuando ya no hace falta.***

---

1. Qué vas a conseguir

Tu personaje, movido con el teclado, dispara gemas hacia donde hagas clic con el ratón. Las gemas vuelan en línea recta y desaparecen al salir de la pantalla.

[CAPTURA 15.1]



"El resultado de la práctica 15: cada clic lanza una gema hacia el cursor."

---

2. Materiales

Descomprime `practica_15_inicio.zip`. Trae el personaje de la práctica 06 y dos imágenes: `gema.png`, que es el proyectil, y `nave.png`, que solo se usa en el reto rojo.

---

3. De qué se compone un disparo

Hoy se juntan tres cosas que ya sabes hacer por separado:

Pieza	De dónde viene
Crear algo desde el código	Práctica 14
Calcular la dirección hacia un punto	Práctica 12
Una zona que detecta	Práctica 07

Lo único nuevo es que el objeto creado **guarda su propia dirección** y se borra solo cuando sale de la pantalla.

---

4. Paso a paso

4.1 La escena del proyectil

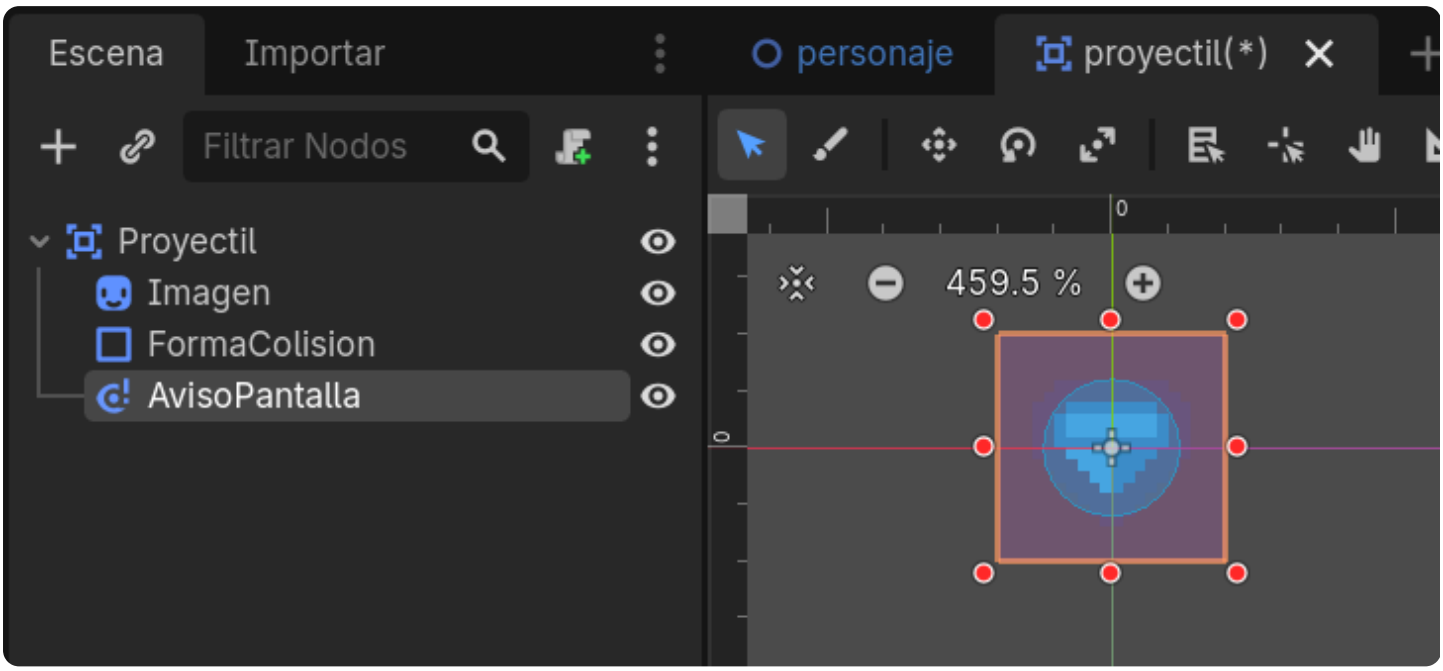
1. **Escena** > **Nueva Escena**. Como raíz elige **Otro Nodo** y busca `Area2D`. Llámalo `Proyectil`.

> *¿Por qué `Area2D` y no un cuerpo? Porque un proyectil tiene que atravesar las cosas y avisar de que las ha tocado, no chocar contra ellas y quedarse parado.*

2. Añádele un `Sprite2D` llamado `Imagen` con la textura `gema.png`.
3. Añádele un `CollisionShape2D` llamado `FormaColision` con una **Nuevo CircleShape2D** de radio `6`.

4. Añádele un tercer hijo: un `VisibleOnScreenNotifier2D` llamado `AvisoPantalla`. Este nodo no se ve al ejecutar: su único trabajo es avisar cuando su dueño entra o sale de la pantalla.

[CAPTURA 15.2]



"El proyectil con un área de colisión y un aviso de fuera de pantalla."

5. Añade a `Proyectil` un script en `res://scripts/proyectil.gd`:

```
extends Area2D

var direccion := Vector2.RIGHT
var velocidad := 220.0


func _physics_process(delta: float) -> void:
    position += direccion * velocidad * delta
```

6. Conecta la señal del aviso: selecciona `AvisoPantalla`, panel **Nodo**, pestaña **Señales**, doble clic en `screen_exited`. Conéctala a `Proyectil` con el método `_on_screen_exited`, y complétala:

```
func _on_screen_exited() -> void:
    queue_free()
```

7. Guarda como `escenas/proyectil.tscn`.

4.2 Que el personaje dispare

8. Abre tu escena principal y guárdala como `escenas/mundo.tscn` .
9. Abre `scripts/personaje.gd` y déjalo así:

```
extends CharacterBody2D

const ESCENA_PROYECTIL := preload("res://escenas/proyectil.tscn")

var velocidad := 100.0


func _physics_process(_delta: float) -> void:
    var direccion := Input.get_vector(
        "mover_izquierda", "mover_derecha", "mover_arriba", "mover_abajo")
    velocity = direccion * velocidad
    move_and_slide()

    if Input.is_action_just_pressed("disparar"):
        disparar()


func disparar() -> void:
    var proyectil := ESCENA_PROYECTIL.instantiate()
    proyectil.global_position = global_position
    proyectil.direccion = global_position.direction_to(get_global_mouse_position())
    get_parent().add_child(proyectil)
```

10. Ejecuta con **F6**, muévete y haz clic por la pantalla.

---

5. El código, línea a línea

`Input.is_action_just_pressed("disparar")` La acción `disparar` está en el Mapa de Entrada del curso desde el primer día, asignada al **clic izquierdo del ratón**. Ábrelo y compruébalo.

Se usa `just_pressed` y no `pressed` : si no, con el botón mantenido saldrían sesenta gemas por segundo.

`func disparar() -> void:` Es la primera función que escribes **tú**, con el nombre que quieras. Las anteriores (`_process` , `_ready` , `_on_...`) las llamaba Godot o una señal. A esta la llamas tú desde la línea de arriba.

Podría estar todo dentro del `if`, pero separarlo en una función con nombre hace el código mucho más fácil de leer.

`proyectil.global_position = global_position` Coloca la gema donde está el personaje. Fíjate en que se usa `global_position` en los dos lados: el proyectil va a colgar de `Mundo`, y el personaje también, pero conviene coger la costumbre.

`proyectil.direccion = ...` Aquí está la idea central de la práctica. El proyectil tiene una variable `direccion` declarada en su propio script, y **desde fuera se le escribe un valor**.

Es la primera vez que un script le cambia una variable a otro objeto. A partir de aquí, el proyectil vuela solo y ya no necesita a nadie.

`get_parent().add_child(proyectil)` `get_parent()` devuelve el nodo padre, que aquí es `Mundo`. La gema se cuelga de `Mundo`, **no del personaje**.

> ***Esto es importante y es el error más gracioso de la práctica. Si escribes `add_child(proyectil)` a secas, la gema se cuelga del personaje. Y como los hijos se mueven con su padre (lo viste en la práctica 06), las gemas te seguirían por la pantalla como un enjambre en vez de salir volando. Pruébalo un momento: merece la pena verlo.***

`screen_exited` La señal del `VisibleOnScreenNotifier2D`. Sin ella, cada gema seguiría volando para siempre hacia el infinito, consumiendo recursos aunque nadie la vea.

Es la solución al problema que dejó abierto la práctica 14: **lo que se crea, hay que borrarlo**.

---

6. Pruébalo

- [ ] Cada clic lanza una gema.
- [ ] La gema sale hacia el punto donde has hecho clic, no siempre en la misma dirección.
- [ ] Las gemas **no te siguen**: una vez lanzadas van a su aire.
- [ ] Puedes moverte y disparar a la vez.
- [ ] Manteniendo el botón pulsado solo sale una gema.
- [ ] En el panel **Salida** no hay nada en rojo.

---

7. La teoría en cinco líneas

- Un proyectil es un `Area2D` : atraviesa y avisa, no choca.
- Lo instanciado se puede **configurar desde fuera** antes de meterlo en el juego.
- `get_parent().add_child()` cuelga el objeto del mundo, no de quien lo dispara.
- Un hijo se mueve con su padre, así que un disparo nunca debe ser hijo del que dispara.
- Todo lo que se crea en marcha tiene que **borrarse solo** cuando deja de hacer falta.

---

8. Si no te funciona

Lo que ves	Qué pasa	Cómo se arregla
Las gemas te siguen	`add_child` sin `get_parent()`	Deben colgar de `Mundo`
No sale nada	Falta `add_child`	Como en la práctica 14
Salen sesenta por segundo	Usaron `is_action_pressed`	Tiene que ser `just_pressed`
Todas van a la derecha	No se asigna `direccion`	Es la línea del `direction_to`
Salen desde la esquina	No se asigna la posición	`proyectil.global_position = global_position`
Se acumulan y el juego va lento	Falta conectar `screen_exited`	Icono verde en el panel Nodo
`Invalid assignment ... 'direccion'`	El script del proyectil no tiene esa variable	Tiene que estar declarada arriba
Al hacer clic no pasa nada	La acción `disparar` no existe en este proyecto	Mapa de Entrada

---

9. Retos

Verde · Cambia `velocidad` a `60` y luego a `600` . Hay que tener cuidado con los valores muy altos, pues la gema **puede atravesar cosas** sin llegar a tocarlas, porque entre un fotograma y el siguiente ha saltado por encima.

Amarillo · Pon una **cadencia de disparo**: que no se pueda disparar más de dos veces por segundo aunque se aporree el botón. Pista: un `Timer` como el de la práctica 14, o una variable que recuerde si ya se puede volver a disparar.

Rojo · Que el proyectil **apunte hacia donde vuela**. Cambia la imagen por `nave.png` y gíralo al crearlo. Pista: un vector sabe decir su propio ángulo con `angle()`, y un nodo tiene la propiedad `rotation`. Recuerda por qué `nave.png` está dibujada mirando a la derecha.

---

10. Autoevaluación

- [ ] Sé por qué un proyectil es un `Area2D` y no un cuerpo.
- [ ] Sé configurar un objeto instanciado antes de meterlo en la escena.
- [ ] Sé explicar por qué el disparo no puede ser hijo del que dispara.
- [ ] Sé usar `get_parent()`.
- [ ] Sé escribir una función propia y llamarla.
- [ ] Sé hacer que un objeto se destruya al salir de la pantalla.
- [ ] Sé por qué aquí va `just_pressed` y no `pressed`.

---

11. Vocabulario

Español	Inglés	Qué es
Proyectil	Projectile	Lo que se lanza
Aviso de pantalla	VisibleOnScreenNotifier2D	Nodo que avisa al entrar o salir de la vista
Cadencia	Fire rate, cooldown	Cada cuánto se puede volver a disparar
Nodo padre	Parent node	El nodo del que cuelga otro

---

12. Entrega

Comprime la carpeta como `apellido_practica15.zip` y súbela al Aula Virtual de Alexia.

Capturas necesarias en esta práctica

Código	Contenido
15.1	Resultado final, a ser posible en GIF
15.2	Escena del proyectil con sus cuatro nodos